

Banking Transactions API

User Guide & Run Book

One REST contract · 4 backends (TDD) · 3 frontends · one launcher

Project	gen-ai-software-engineering / homework-1
Author	Olena Chiperi
Built with	Claude (AI-assisted, test-first)
Document	Detailed run book with live console output + Chrome screenshots

Contents

1 What this project is	2
Backends — same contract, four stacks	2
Frontends — same API, fully decoupled	2
2 Architecture & API contract	3
Shared rules every backend follows	3
3 Prerequisites	4
4 Quick start — one command	4
5 Variation 1 — FastAPI backend + Angular web frontend	6
5.1 Start the backend from the console	6
5.2 The API responding (captured live)	6
5.3 Start the Angular frontend	6
5.4 The app in the browser	6
5.5 What to try in the UI	9
6 Variation 2 — FastAPI backend + Electron desktop frontend	10
6.1 Run it	10
6.2 The launcher and the desktop app together	10
7 API reference (with live examples)	11
7.1 Endpoints	11
7.2 Create — success (201)	11
7.3 Create — validation failure (400, structured)	11
7.4 Filtering and a missing id	11
7.5 CSV export	12
7.6 Interactive docs & health in the browser	12
8 Running the test suites	14
9 Troubleshooting	14

1 What this project is

This project is a **Banking Transactions REST API** built **test-first (TDD)** with AI assistance. The distinctive part: from a single engineered prompt and one TDD plan, the **same HTTP contract** was implemented in four backends and consumed by three frontends. Any frontend works against any backend, because they all speak identical REST/JSON on **port 3000**.

Backends — same contract, four stacks

Stack	Folder	Tests	Notes
Java 17 + Spring Boot 3	backend/	53	Primary, fully documented; Bean Validation, Actuator, Scalar
Go 1.22 + Chi	backend-go/	50	shopspring/decimal, RWMutex store, hand-rolled validation
Python + FastAPI	backend-fastapi/ /	32	Pydantic v2, native Swagger /docs + /openapi.json
C# / .NET 10 Minimal API	backend-dotnet/ /	18	System.Text.Json, xUnit, built-in OpenAPI + Scalar

Total: **153 automated backend tests**, all green. Money is always exact (BigDecimal / decimal / shopspring — never float); storage is in-memory (no database).

Frontends — same API, fully decoupled

Client	Stack	Folder
Web app (primary)	Angular 22 + Microsoft Fluent (glass UI, light/dark)	frontend/
Desktop	Electron (Chromium + Node)	frontend-electron/
Web (mobile-friendly)	Flutter Web (Dart, Material 3)	frontend-flutter/

This guide documents two runnable variations

Section 5 runs **FastAPI + the Angular web app** (browser); Section 6 runs **FastAPI + the Electron desktop app**. The console output shown throughout was captured from a real run of the FastAPI backend; the UI screenshots are from Chrome / the desktop app.

2 Architecture & API contract

The system is a classic client–server split. A stateless REST API on port 3000 holds transactions in an in-memory store and exposes them over JSON. Each frontend is a thin client that renders the data and posts new transactions — no business logic is duplicated in the UI.

```
Frontend (Angular / Electron / Flutter)
  | HTTP + JSON (proxy /api -> :3000, or direct)
  v
REST API http://localhost:3000
  /transactions          POST create (201) / GET list (+filters)
  /transactions/{id}     GET one (404 if missing)
  /transactions/export    GET CSV (text/csv attachment)
  /accounts/{id}/balance GET balance
  /accounts/{id}/summary GET deposits/withdrawals/count/last date
  /accounts/{id}/interest GET simple interest (rate, days)
  /actuator/health        GET liveness -> {"status":"UP"}
  /docs + /openapi.json   interactive API reference (Swagger / Scalar)
  |
  v
In-memory store (thread-safe) – reseed with demo/seed.sh after each restart
```

Shared rules every backend follows

- Enums are lowercase in JSON (deposit, withdrawal, transfer).
- Validation failures return **400** with a structured body: { error, details:[{field, message}]}.
- Amounts must be positive with ≤ 2 decimals; accounts match ACC-XXXXX; currency must be ISO 4217.
- Health at /actuator/health and API docs at /docs exist in every backend.

3 Prerequisites

Install only the toolchain for the stack you want to run. The launcher checks and starts the rest.

To run...	You need
Any backend + a browser frontend	A modern browser (Chrome recommended)
Java backend	JDK 17+ and Maven
Go backend	Go 1.22+
FastAPI backend	Python 3.11+ (the launcher creates a venv and pip-installs deps)
.NET backend	.NET SDK 10
Angular web frontend	Node.js 18+ (installs from npm on first run)
Electron desktop	Node.js 18+ (downloads the Electron runtime on first npm install)
Flutter web	Flutter SDK 3.4+ with web enabled, plus Chrome

Ports used

The API always listens on **3000**; the Angular dev server on **4200**. The launcher frees both on exit (Ctrl+C).

4 Quick start — one command

From the `homework-1/` folder, the interactive launcher lets you pick a backend, then a frontend, and runs the chosen combination. It starts the backend on port 3000, waits until it is healthy, seeds sample data, and launches the frontend. **Ctrl+C** stops everything and frees the ports.

```
cd homework-1
./run.sh
```

What the prompts look like

```
Choose a backend:
 1) Java + Spring Boot
 2) Go + Chi
 3) Python + FastAPI
 4) C# / .NET
Backend [1-4]: 3

Starting backend: Python + FastAPI (logs: .logs/backend.log)
Waiting for API on :3000 ..... - ready
Seeding sample data...

Choose a frontend:
 1) Angular (web, http://localhost:4200)
 2) Electron (desktop window)
 3) Flutter Web (opens Chrome)
 4) None (backend only)
Frontend [1-4]: 1

Backend running: http://localhost:3000 (docs: /docs, health: /actuator/health)
Press Ctrl+C to stop everything.
```

Prefer to run parts by hand?

```
# Backend (pick one)
cd backend      && mvn spring-boot:run
cd backend-go   && go run ./cmd/server
cd backend-fastapi && uvicorn app.main:app --port 3000
cd backend-dotnet && dotnet run --project src/BankingApi

# Frontend (pick one, in a second terminal)
cd frontend      && npm install && npm start      # Angular -> :4200
cd frontend-electron && npm install && npm start    # Electron desktop
cd frontend-flutter && flutter run -d chrome       # Flutter web

# Sample data (any backend, after it is up)
bash demo/seed.sh
```

5 Variation 1 — FastAPI backend + Angular web frontend

This is the primary web experience: the Python/FastAPI backend serving the Angular Fluent UI in the browser. Below is the **real console session** from starting the backend and exercising the API, followed by **Chrome screenshots** of the running app.

5.1 Start the backend from the console

```
cd homework-1/backend-fastapi
uvicorn app.main:app --port 3000
```

Verify it is healthy, then seed demo data:

```
$ curl -s http://localhost:3000/actuator/health
{"status":"UP"}

$ bash demo/seed.sh
✓ {"fromAccount":"ACC-000000","toAccount":"ACC-12345","amount":500.0,...}
✓ {"fromAccount":"ACC-55512","toAccount":"ACC-000000","amount":120.25,...}
Done. Open http://localhost:4200 (or GET http://localhost:3000/transactions).
```

5.2 The API responding (captured live)

A few representative calls against the seeded store:

```
$ curl -s http://localhost:3000/accounts/ACC-12345/balance
{"accountId":"ACC-12345","balance":1525.25}

$ curl -s http://localhost:3000/accounts/ACC-12345/summary
{"totalDeposits":1750.75,"totalWithdrawals":50.0,
 "transactionCount":5,"mostRecentTransactionDate":"2026-07-02T18:15:29Z"}

$ curl -s 'http://localhost:3000/accounts/ACC-12345/interest?rate=0.05&days=30'
{"accountId":"ACC-12345","balance":1525.25,"rate":0.05,"days":30,"interest":6.27}
```

5.3 Start the Angular frontend

```
cd homework-1/frontend
npm install      # first run only
npm start       # dev server on http://localhost:4200
```

The dev server proxies `/api` to the backend on port 3000, so no CORS setup is needed. Open `http://localhost:4200` in Chrome.

5.4 The app in the browser

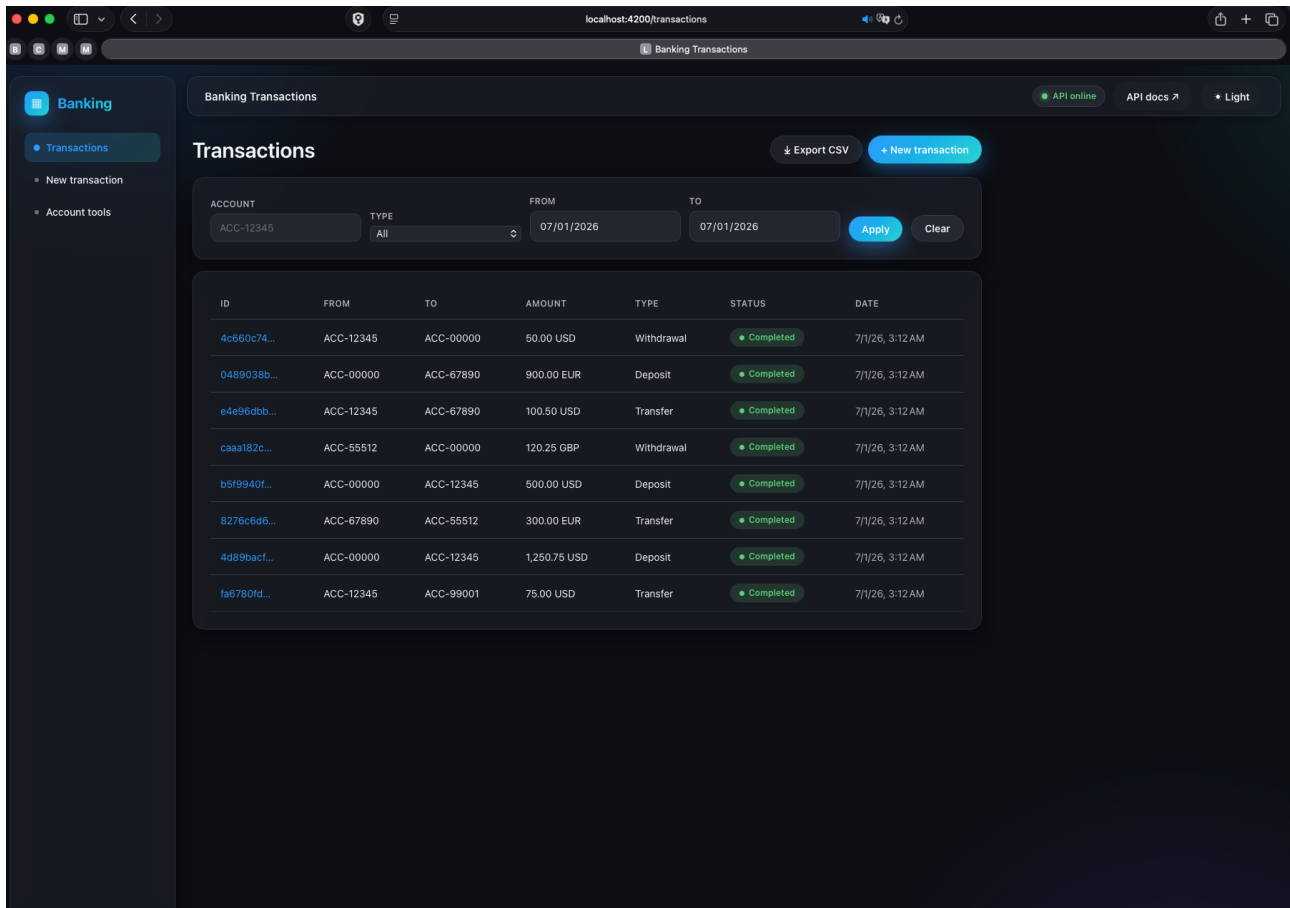


Figure 5-1. Transactions list with filters, status badges, and the live “API online” chip — dark theme (Chrome).

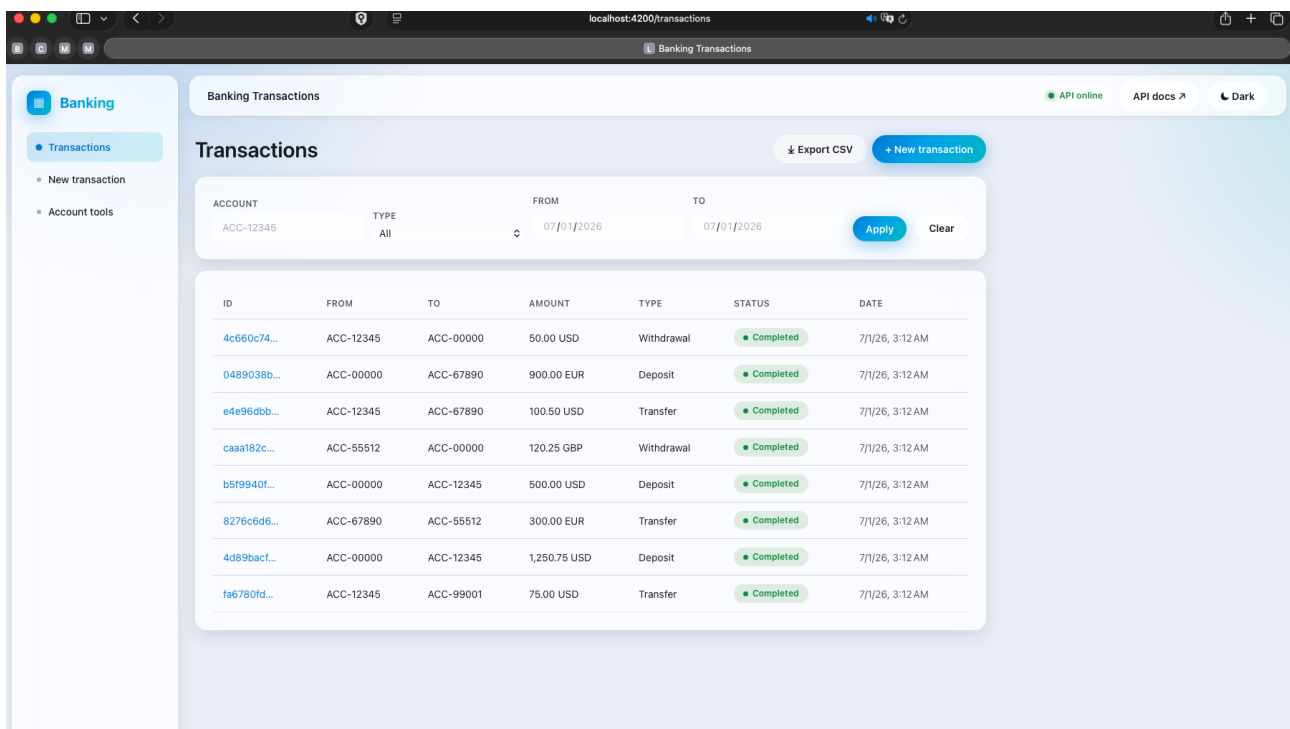


Figure 5-2. The same transactions view in the light theme (Chrome).

Banking Transactions

API online API docs Dark

New transaction

FROM ACCOUNT TO ACCOUNT

ACC-12345 ACC-67890

Must match ACC-XXXXX. Must match ACC-XXXXX.

AMOUNT CURRENCY

100.50 USD

Enter a positive amount (max 2 decimals).

TYPE

Transfer

Create transaction Cancel

Figure 5-3. “New transaction” form: an invalid account format and a non-positive amount are flagged inline, mapped from the backend’s 400 details[].

Banking Transactions

API online API docs Dark

Account tools

Balance

ACCOUNT

ACC-12345 Get balance

Summary

ACCOUNT

ACC-12345 Get summary

Simple interest

ACCOUNT RATE DAYS

ACC-12345 0.05 30 Calculate

Figure 5-4. Account tools — balance, summary, and the simple-interest calculator, each calling the matching endpoint.

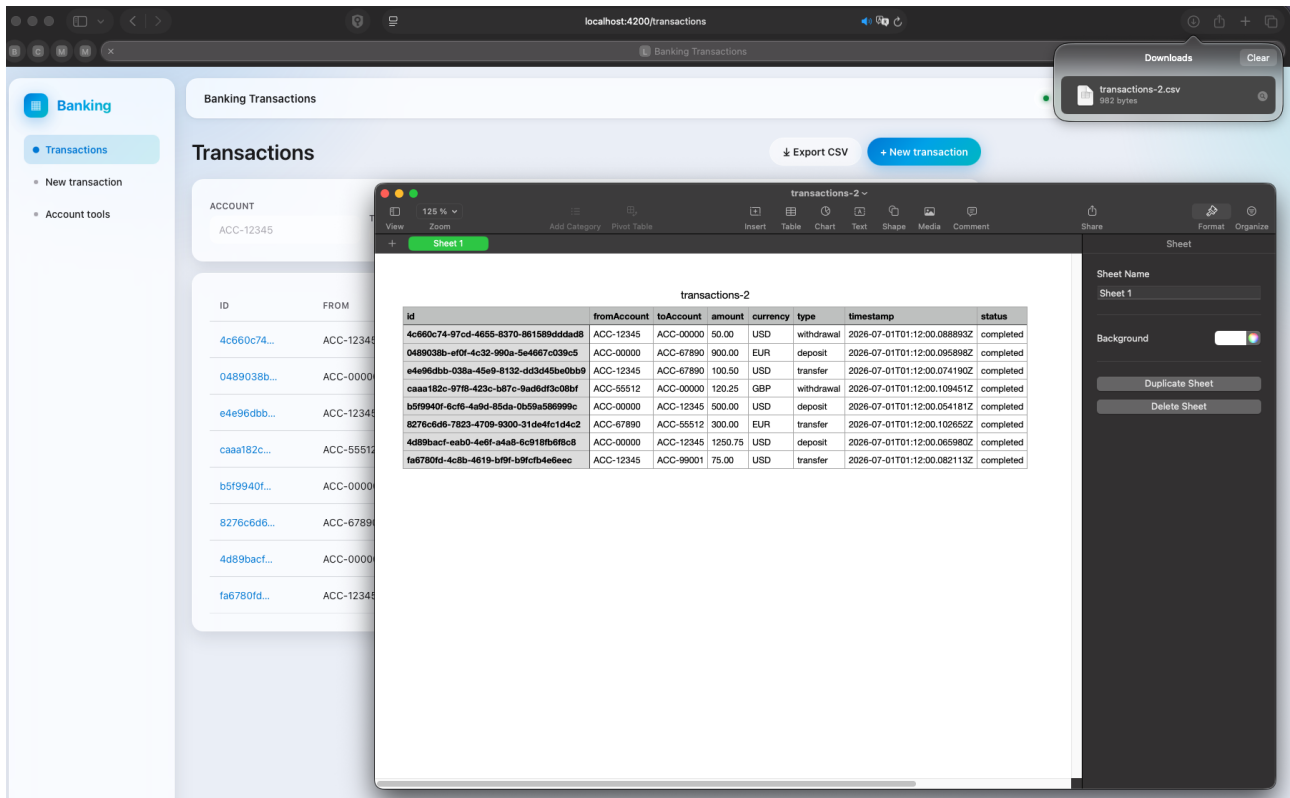


Figure 5-5. “Export CSV” downloads the transactions (text/csv attachment) and opens them in a spreadsheet.

5.5 What to try in the UI

- Filter the list by account, type, or date range — the filters combine (AND) and map to query params on GET /transactions.
- Submit the create form with bad values to see per-field validation from the backend.
- Toggle light/dark from the top bar; the API-status chip polls /actuator/health.
- Click “Export CSV” to download the current transactions.

6 Variation 2 — FastAPI backend + Electron desktop frontend

Same backend, different client: the Electron app renders the identical Fluent UI in a native desktop window instead of a browser tab. It calls the API directly (the window is created with web security relaxed for the local demo), so no proxy is required.

6.1 Run it

The fastest path is the launcher — choose backend **3 (FastAPI)**, then frontend **2 (Electron)**:

```
cd homework-1
./run.sh          # backend: 3 (FastAPI)  frontend: 2 (Electron)
```

Or by hand, with a backend already running on port 3000:

```
cd homework-1/frontend-electron
npm install      # downloads the Electron runtime (first run)
npm start        # opens the desktop window
```

6.2 The launcher and the desktop app together

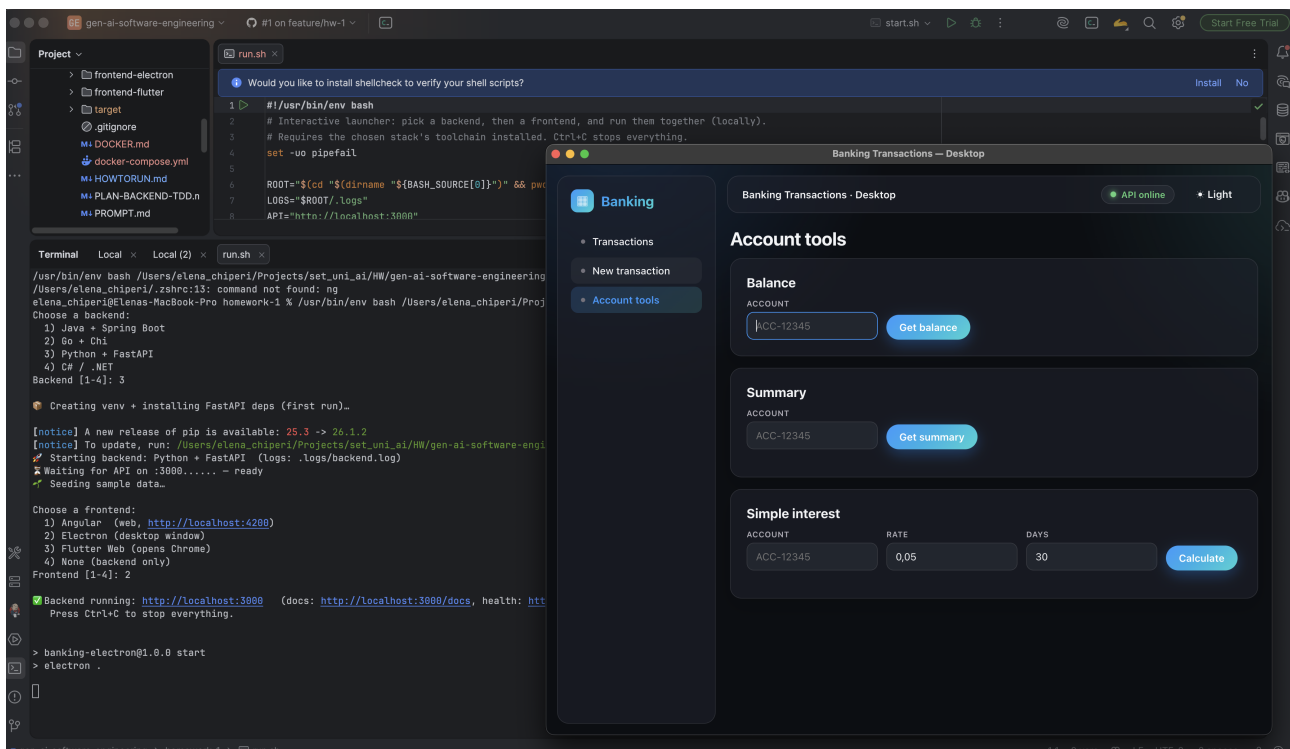


Figure 6-1. The end-to-end result: `./run.sh` started the FastAPI backend (seeded, “API online”) and launched the Electron desktop app — here on the Account tools page. The same launcher runs any backend with any frontend.

Flutter is the third option

Choosing frontend **3** in the launcher opens the Flutter Web client in Chrome instead. It offers the same features (list, filters, create with validation, account tools, CSV, light/dark) and also talks to port 3000.

7 API reference (with live examples)

Every example below is real output captured from the running FastAPI backend. The other backends return byte-for-byte the same shapes.

7.1 Endpoints

Method & path	Purpose
POST /transactions	Create a transaction (server assigns id, timestamp, status) → 201
GET /transactions	List all; optional filters: accountId, type, from, to (combinable)
GET /transactions/{id}	Fetch one; 404 if not found
GET /transactions/export	Download all as CSV (text/csv attachment)
GET /accounts/{id}/balance	Net balance for an account
GET /accounts/{id}/summary	Deposits, withdrawals, count, most-recent date
GET /accounts/{id}/interest	Simple interest (query: rate, days)
GET /actuator/health	Liveness → {"status":"UP"}
GET /docs, /openapi.json	Interactive API reference + OpenAPI spec

7.2 Create — success (201)

```
$ curl -s -X POST http://localhost:3000/transactions -H 'Content-Type: application/json' \
  -d '{"fromAccount":"ACC-000000","toAccount":"ACC-12345",
    "amount":250.50,"currency":"USD","type":"deposit"}'

{"id":"9e1bef8d-9641-459c-85e4-b27d8fdaca98","fromAccount":"ACC-000000",
  "toAccount":"ACC-12345","amount":250.5,"currency":"USD","type":"deposit",
  "timestamp":"2026-07-02T18:22:54Z","status":"completed"}
```

7.3 Create — validation failure (400, structured)

```
$ curl -s -X POST http://localhost:3000/transactions -H 'Content-Type: application/json' \
  -d '{"fromAccount":"BAD","toAccount":"ACC-12345",
    "amount":-5,"currency":"US","type":"gift"}'

{"error":"Validation failed","details":[
  {"field":"fromAccount","message":"String should match pattern '^ACC-[A-Za-z0-9]+$'"},
  {"field":"amount","message":"Input should be greater than 0"},
  {"field":"currency","message":"Value error, Invalid currency code"},
  {"field":"type","message":"Input should be 'deposit','withdrawal' or 'transfer'"}]}
```

7.4 Filtering and a missing id

```
$ curl -s 'http://localhost:3000/transactions?type=deposit' # -> 4 deposits
$ curl -s 'http://localhost:3000/transactions?accountId=ACC-12345' # -> 6 txns (from OR to)

$ curl -s -o /dev/null -w '%{http_code}' http://localhost:3000/transactions/nope
404
```

7.5 CSV export

```
$ curl -s -D - http://localhost:3000/transactions/export -o out.csv
content-type: text/csv; charset=utf-8
content-disposition: attachment; filename="transactions.csv"

id,fromAccount,toAccount,amount,currency,type,timestamp,status
03bcdfe9-...,ACC-00000,ACC-12345,500.0,USD,deposit,2026-07-02T18:23:09Z,completed
b12bbbe6-...,ACC-00000,ACC-12345,1250.75,USD,deposit,2026-07-02T18:23:09Z,completed
```

7.6 Interactive docs & health in the browser

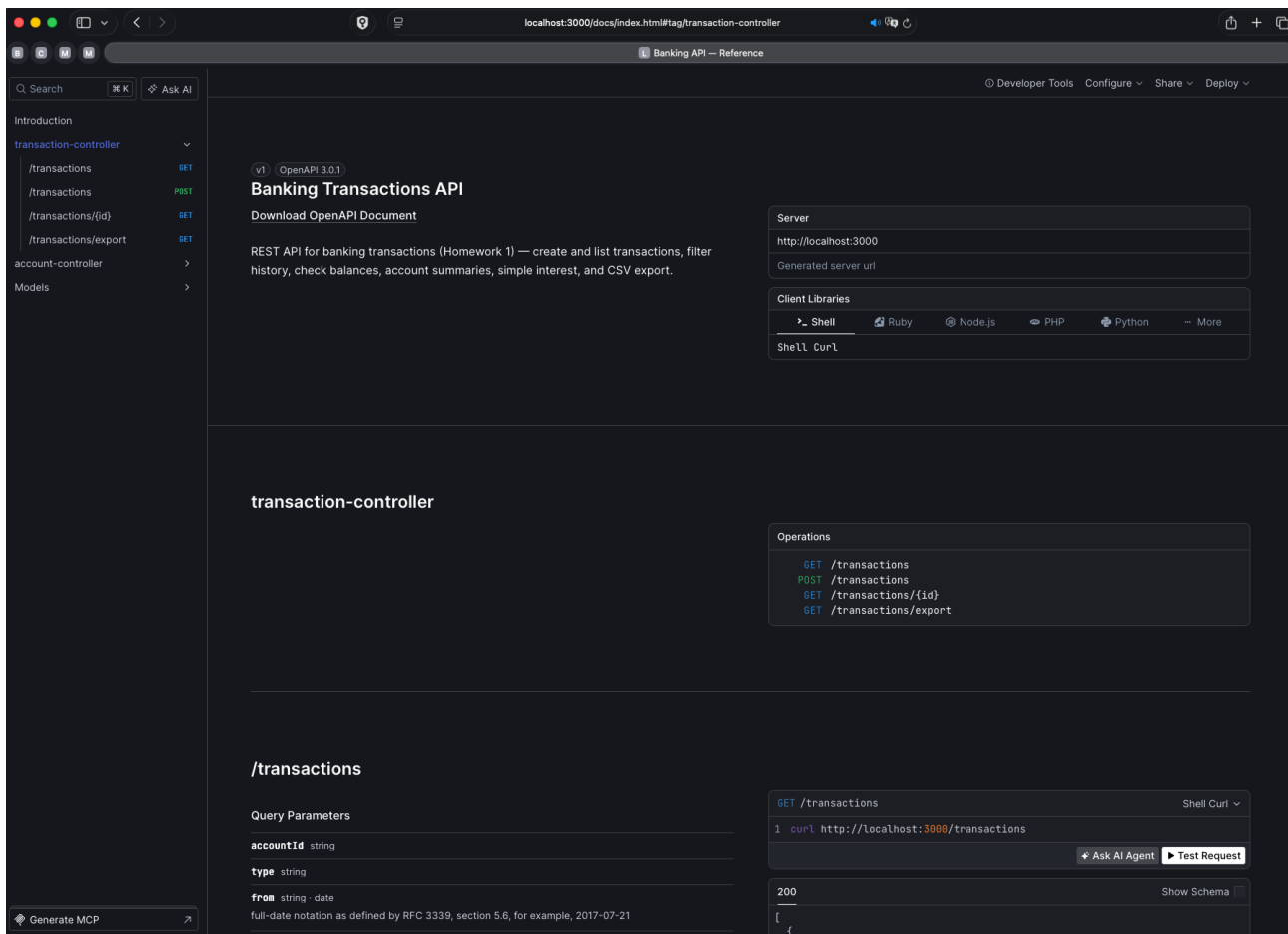


Figure 7-1. The interactive API reference at /docs (rendered from the OpenAPI spec) — open in Chrome.

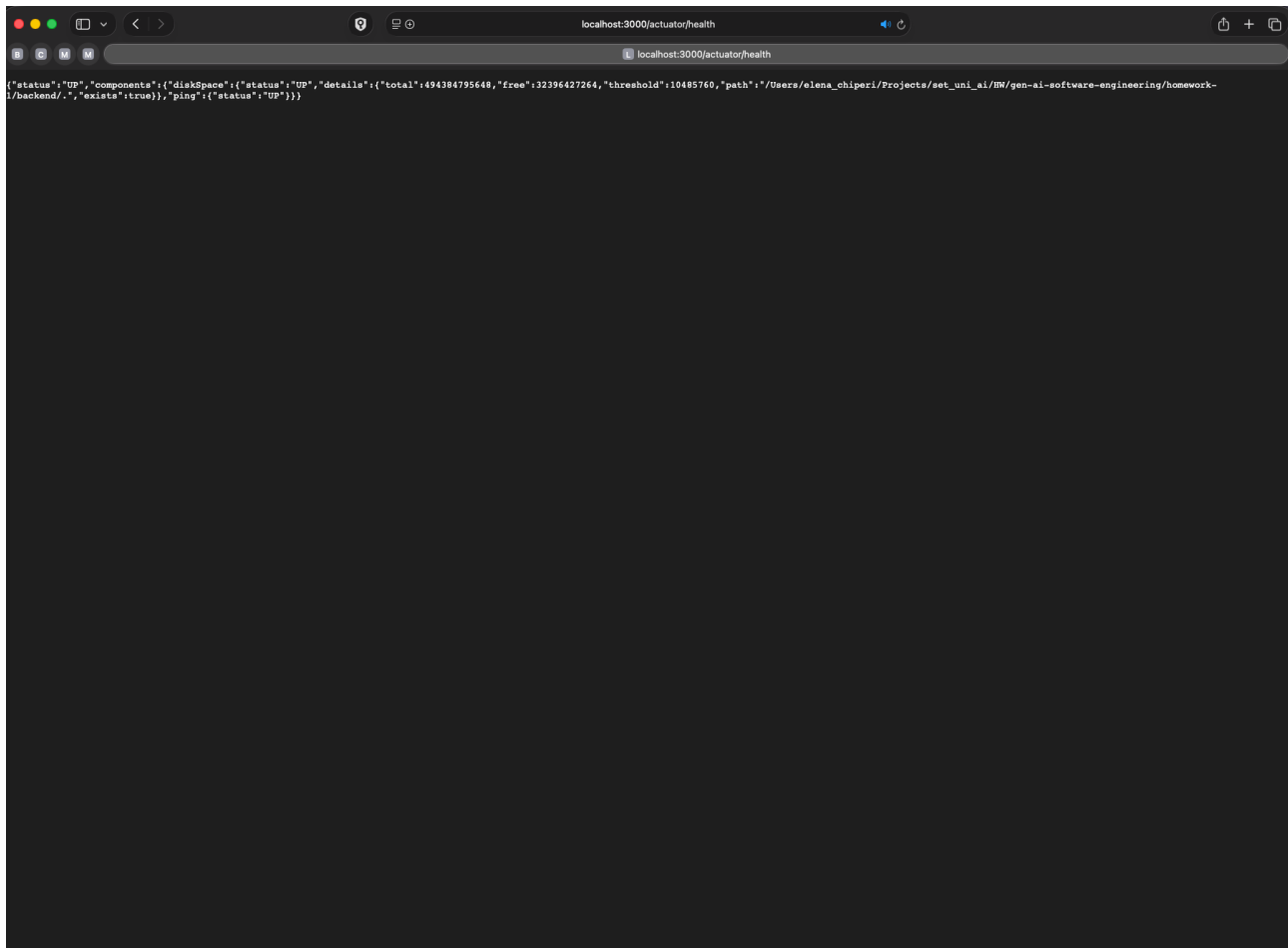


Figure 7-2. GET /actuator/health returning {"status":"UP"} in the browser.

8 Running the test suites

Each backend was built test-first and ships its own suite. Run whichever toolchain you have installed:

```
cd backend      && mvn test      # 53 tests (Java)
cd backend-go   && go test ./... # 50 tests (Go)
cd backend-fastapi && pytest -q  # 32 tests (FastAPI)
cd backend-dotnet && dotnet test  # 18 tests (.NET)
```

Total: **153 tests**, all green. The FastAPI suite (32 tests) was executed in the generation environment; the others were verified locally.

9 Troubleshooting

Symptom	Fix
"API offline" chip / calls fail	Make sure a backend is running on port 3000 (curl /actuator/health). Restart with ./run.sh.
List is empty after restart	The store is in-memory and clears on restart — re-run bash demo/seed.sh.
Port 3000 or 4200 already in use	Stop the previous run (Ctrl+C). If it lingers, kill the process holding the port, then retry.
Angular: browser calls blocked by CORS	Use npm start (dev proxy /api→:3000). Don't open the built files directly without the proxy.
Electron / Flutter: CORS errors	These call :3000 directly; the demo relaxes web security locally. Keep using the provided run commands.
ng: command not found	Run via npm start (uses the local CLI) rather than a global ng, or install deps with npm install first.
Flutter: missing platform files	Run flutter create . --platforms=web once, then restore lib/main.dart from git if it was overwritten.

How this guide was produced

The console output and API responses shown here were captured from a real run of the FastAPI backend on port 3000. Browser screenshots come from Chrome / the Electron desktop app. A live browser could not be launched inside the build environment, so the UI figures reuse the project's captured Chrome screenshots rather than newly rendered ones.